

LightFM

```
class lightfm.LightFM(no_components=10, k=5, n=10, learning_schedule='adagrad',  
loss='logistic', learning_rate=0.05, rho=0.95, epsilon=1e-06, item_alpha=0.0, user_alpha=0.0,  
max_sampled=10, random_state=None) \[source\]
```

A hybrid latent representation recommender model.

The model learns embeddings (latent representations in a high-dimensional space) for users and items in a way that encodes user preferences over items. When multiplied together, these representations produce scores for every item for a given user; items scored highly are more likely to be interesting to the user.

The user and item representations are expressed in terms of representations of their features: an embedding is estimated for every feature, and these features are then summed together to arrive at representations for users and items. For example, if the movie 'Wizard of Oz' is described by the following features: 'musical fantasy', 'Judy Garland', and 'Wizard of Oz', then its embedding will be given by taking the features' embeddings and adding them together. The same applies to user features.

The embeddings are learned through [stochastic gradient descent](#) methods.

Four loss functions are available:

- logistic: useful when both positive (1) and negative (-1) interactions are present.
- BPR: Bayesian Personalised Ranking ¹ pairwise loss. Maximises the prediction difference between a positive example and a randomly chosen negative example. Useful when only positive interactions are present and optimising ROC AUC is desired.
- WARP: Weighted Approximate-Rank Pairwise ² loss. Maximises the rank of positive examples by repeatedly sampling negative examples until rank violating one is found. Useful when only positive interactions are present and optimising the top of the recommendation list (precision@k) is desired.
- k-OS WARP: k-th order statistic loss ³. A modification of WARP that uses the k-th positive example for any given user as a basis for pairwise updates.

Two learning rate schedules are available:

- adagrad: ⁴
- adadelta: ⁵

Parameters: • `no_components` (*int, optional*) – the dimensionality of the

Parameters:

- **n_components** (*int, optional*) – the dimensionality of the feature latent embeddings.
- **k** (*int, optional*) – for k-OS training, the k-th positive example will be selected from the n positive examples sampled for every user.
- **n** (*int, optional*) – for k-OS training, maximum number of positives sampled for each update.
- **learning_schedule** (*string, optional*) – one of ('adagrad', 'adadelata').
- **loss** (*string, optional*) – one of ('logistic', 'bpr', 'warp', 'warp-kos'): the loss function.
- **learning_rate** (*float, optional*) – initial learning rate for the adagrad learning schedule.
- **rho** (*float, optional*) – moving average coefficient for the adadelata learning schedule.
- **epsilon** (*float, optional*) – conditioning parameter for the adadelata learning schedule.
- **item_alpha** (*float, optional*) – L2 penalty on item features. Tip: setting this number too high can slow down training. One good way to check is if the final weights in the embeddings turned out to be mostly zero. The same idea applies to the user_alpha parameter.
- **user_alpha** (*float, optional*) – L2 penalty on user features.
- **max_sampled** (*int, optional*) – maximum number of negative samples used during WARP fitting. It requires a lot of sampling to find negative triplets for users that are already well represented by the model; this can lead to very long training times and overfitting. Setting this to a higher number will generally lead to longer training times, but may in some cases improve accuracy.
- **random_state** (*int seed, RandomState instance, or None*) – The seed of the pseudo random number generator to use when shuffling the data and initializing the parameters.

Variables:

- **~LightFM.item_embeddings** (*np.float32 array of shape [n_item_features, n_components]*) – Contains the estimated latent vectors for item features. The [i, j]-th entry gives the value of the j-th component for the i-th item feature. In the simplest case where the item feature matrix is an identity matrix, the i-th row will represent the i-th item latent vector.
- **~LightFM.user_embeddings** (*np.float32 array of shape [n_user_features, n_components]*) – Contains the estimated latent vectors for user features. The [i, j]-th entry gives the

value of the j -th component for the i -th user feature. In the simplest case where the user feature matrix is an identity matrix, the i -th row will represent the i -th user latent vector.

- `~LightFM.item_biases` (*np.float32 array of shape $[n_item_features,]$*) – Contains the biases for item_features.
- `~LightFM.user_biases` (*np.float32 array of shape $[n_user_features,]$*) – Contains the biases for user_features.

Notes

Users' and items' latent representations are expressed in terms of their features' representations. If no feature matrices are provided to the `lightfm.LightFM.fit()` or `lightfm.LightFM.predict()` methods, they are implicitly assumed to be identity matrices: that is, each user and item are characterised by one feature that is unique to that user (or item). In this case, LightFM reduces to a traditional collaborative filtering matrix factorization method.

For including features, there are two strategies:

1. Characterizing each user/item *only* by its features.
2. Characterizing each user/item by its features *and* an identity matrix that captures interactions between users and items directly.

1. When using only features, the feature matrix should be of shape `(num_<users/items> x num_features)`. To build these feature matrices, it is recommended to use the build methods from the class `lightfm.data.Dataset` and setting the `<user/item>_identity_features` to `False`. An embedding will then be estimated for every feature: that is, there will be `num_features` embeddings. To obtain the representation for user i , the model will look up the i -th row of the feature matrix to find the features with non-zero weights in that row; the embeddings for these features will then be added together to arrive at the user representation. For example, if user 10 has weight 1 in the 5th column of the user feature matrix, and weight 3 in the 20th column, that user's representation will be found by adding together the embedding for the 5th and the 20th features (multiplying the latter by 3). The same goes for items.

Note: This strategy may result in a less expressive model because no per-user features are estimated, the model may underfit. To combat this, follow strategy 2. and include per-user (per-item) features (that is, an identity matrix) as part of the feature matrix.

2. To use features alongside user-item interactions, the feature matrix should include an identity matrix. The resulting feature matrix should be of shape `(num_<users/items> x (num_<users/items> + num_features))`. This strategy is the default when using the `Lightfm.data.Dataset` class. The behavior is controlled by the `<user/item>_identity_features=True` default arguments.

References

- [1] : Rendle, Steffen, et al. "BPR: Bayesian personalized ranking from implicit feedback." Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence. AUAI Press, 2009.
- [2] : Weston, Jason, Samy Bengio, and Nicolas Usunier. "Wsabie: Scaling up to large vocabulary image annotation." IJCAI. Vol. 11. 2011.
- [3] : Weston, Jason, Hector Yee, and Ron J. Weiss. "Learning to rank recommendations with the k-order statistic loss." Proceedings of the 7th ACM conference on Recommender systems. ACM, 2013.
- [4] : Duchi, John, Elad Hazan, and Yoram Singer. "Adaptive subgradient methods for online learning and stochastic optimization." The Journal of Machine Learning Research 12 (2011): 2121-2159.
- [5] : Zeiler, Matthew D. "ADADELTA: An adaptive learning rate method." arXiv preprint arXiv:1212.5701 (2012).

fit(interactions, user_features=None, item_features=None, sample_weight=None, epochs=1, num_threads=1, verbose=False) [\[source\]](#)

Fit the model.

For details on how to use feature matrices, see the documentation on the `lightfm.LightFM` class.

- Parameters:**
- **interactions** (*np.float32 coo_matrix of shape [n_users, n_items]*) – the matrix containing user-item interactions. Will be converted to `numpy.float32` dtype if it is not of that type.
 - **user_features** (*np.float32 csr_matrix of shape [n_users, n_user_features], optional*) – Each row contains that user's weights over features.
 - **item_features** (*np.float32 csr_matrix of shape [n_items, n_item_features], optional*) – Each row contains that item's weights over features.

- **sample_weight** (*np.float32 coo_matrix of shape [n_users, n_items], optional*) – matrix with entries expressing weights of individual interactions from the interactions matrix. Its row and col arrays must be the same as those of the interactions matrix. For memory efficiency its possible to use the same arrays for both weights and interaction matrices. Defaults to weight 1.0 for all interactions. Not implemented for the k-OS loss.
- **epochs** (*int, optional*) – number of epochs to run
- **num_threads** (*int, optional*) – Number of parallel computation threads to use. Should not be higher than the number of physical cores.
- **verbose** (*bool, optional*) – whether to print progress messages. If *tqdm* is installed, a progress bar will be displayed instead.

Returns: the fitted model

Return type: LightFM instance

fit_partial(*interactions, user_features=None, item_features=None, sample_weight=None, epochs=1, num_threads=1, verbose=False*) [\[source\]](#)

Fit the model.

Fit the model. Unlike fit, repeated calls to this method will cause training to resume from the current model state.

For details on how to use feature matrices, see the documentation on the `lightfm.LightFM` class.

- Parameters:**
- **interactions** (*np.float32 coo_matrix of shape [n_users, n_items]*) – the matrix containing user-item interactions. Will be converted to numpy.float32 dtype if it is not of that type.
 - **user_features** (*np.float32 csr_matrix of shape [n_users, n_user_features], optional*) – Each row contains that user's weights over features.
 - **item_features** (*np.float32 csr_matrix of shape [n_items, n_item_features], optional*) – Each row contains that item's weights over features.
 - **sample_weight** (*np.float32 coo_matrix of shape [n_users, n_items], optional*) – matrix with entries expressing weights of individual interactions from the interactions matrix. Its row and col arrays must be the same as those

of the interactions matrix. For memory efficiency its possible to use the same arrays for both weights and interaction matrices. Defaults to weight 1.0 for all interactions. Not implemented for the k-OS loss.

- **epochs** (*int, optional*) – number of epochs to run
- **num_threads** (*int, optional*) – Number of parallel computation threads to use. Should not be higher than the number of physical cores.
- **verbose** (*bool, optional*) – whether to print progress messages. If *tqdm* is installed, a progress bar will be displayed instead.

Returns: the fitted model

Return type: LightFM instance

get_item_representations(*features=None*) [\[source\]](#)

Get the latent representations for items given model and features.

Parameters: **features** (*np.float32 csr_matrix of shape [n_items, n_item_features], optional*) – Each row contains that item's weights over features. An identity matrix will be used if not supplied.

Returns:
(*np.float32 array of shape n_items,*
np.float32 array of shape [n_items, num_components])

Biases and latent representations for items.

Return type: (item_biases, item_embeddings)

get_params(*deep=True*) [\[source\]](#)

Get parameters for this estimator.

Parameters: **deep** (*boolean, optional*) – If True, will return the parameters for this estimator and contained subobjects that are estimators.

Returns: **params** – Parameter names mapped to their values.

Return type: mapping of string to any

get_user_representations(*features=None*) [\[source\]](#)

Get the latent representations for users given model and features.

Parameters: **features** (*np.float32 csr_matrix of shape [n_users, n_user_features], optional*) – Each row contains that user's weights over features. An identity matrix will be used if not supplied.

Returns: (**np.float32 array of shape n_users**
 np.float32 array of shape [n_users, num_components]

Biases and latent representations for users.

Return type: (user_biases, user_embeddings)

predict(*user_ids, item_ids, item_features=None, user_features=None, num_threads=1*)
[\[source\]](#)

Compute the recommendation score for user-item pairs.

For details on how to use feature matrices, see the documentation on the `Lightfm.LightFM` class.

Parameters:

- **user_ids** (*integer or np.int32 array of shape [n_pairs,]*) – single user id or an array containing the user ids for the user-item pairs for which a prediction is to be computed. Note that these are LightFM's internal id's, i.e. the index of the user in the interaction matrix used for fitting the model.
- **item_ids** (*np.int32 array of shape [n_pairs,]*) – an array containing the item ids for the user-item pairs for which a prediction is to be computed. Note that these are LightFM's internal id's, i.e. the index of the item in the interaction matrix used for fitting the model.
- **user_features** (*np.float32 csr_matrix of shape [n_users, n_user_features], optional*) – Each row contains that user's weights over features.
- **item_features** (*np.float32 csr_matrix of shape [n_items, n_item_features], optional*) – Each row contains that item's weights over features.
- **num_threads** (*int, optional*) – Number of parallel computation threads to use. Should not be higher than the number of physical cores.

Returns: Numpy array containing the recommendation scores for pairs defined by the inputs

defined by the inputs.

Return type: `np.float32` array of shape `[n_pairs,]`

Notes

As indicated above, this method returns an array of scores corresponding to the score assigned by the model to `_pairs` of `inputs_`. Importantly, this means the `i`-th element of the output array corresponds to the score for the `i`-th user-item pair in the input arrays.

Concretely, you should expect the `lfm.predict([0, 1], [8, 9])` to return an array of `np.float32` that may look something like `[0.42 0.31]`, where 0.42 is the score assigned to the user-item pair (0, 8) and 0.31 the score assigned to pair (1, 9) respectively.

In other words, if you wish to generate the score for a few items (e.g. `[7, 8, 9]`) for two users (e.g. `[0, 1]`), a proper way to call this method would be to use `lfm.predict([0, 0, 0, 1, 1, 1], [7, 8, 9, 7, 8, 9])`, and `_not_ lfm.predict([0, 1], [7, 8, 9])` as you may initially expect (this will throw an exception!).

`predict_rank`(`test_interactions`, `train_interactions=None`, `item_features=None`, `user_features=None`, `num_threads=1`, `check_intersections=True`) [\[source\]](#)

Predict the rank of selected interactions. Computes recommendation rankings across all items for every user in interactions and calculates the rank of all non-zero entries in the recommendation ranking, with 0 meaning the top of the list (most recommended) and `n_items - 1` being the end of the list (least recommended).

Performs best when only a handful of interactions need to be evaluated per user. If you need to compute predictions for many items for every user, use the `predict` method instead.

For details on how to use feature matrices, see the documentation on the `Lightfm.LightFM` class.

- Parameters:**
- **`test_interactions`** (`np.float32 csr_matrix` of shape `[n_users, n_items]`) – Non-zero entries denote the user-item pairs whose rank will be computed.
 - **`train_interactions`** (`np.float32 csr_matrix` of shape `[n_users, n_items]`, optional) – Non-zero entries denote the user-item pairs which will be excluded from rank computation. Use to exclude training set interactions from being scored and ranked for evaluation.

- **user_features** (*np.float32 csr_matrix of shape [n_users, n_user_features], optional*) – Each row contains that user's weights over features.
- **item_features** (*np.float32 csr_matrix of shape [n_items, n_item_features], optional*) – Each row contains that item's weights over features.
- **num_threads** (*int, optional*) – Number of parallel computation threads to use. Should not be higher than the number of physical cores.
- **check_intersections** (*bool, optional, True by default,*) – Only relevant when train_interactions are supplied. A flag that signals whether the test and train matrices should be checked for intersections to prevent optimistic ranks / wrong evaluation / bad data split.

Returns: the [i, j]-th entry of the matrix will contain the rank of the j-th item in the sorted recommendations list for the i-th user. The degree of sparsity of this matrix will be equal to that of the input interactions matrix.

Return type: np.float32 csr_matrix of shape [n_users, n_items]

set_params(params)** [\[source\]](#)

Set the parameters of this estimator.

Return type: self